

ANGULAR HANDS-ON

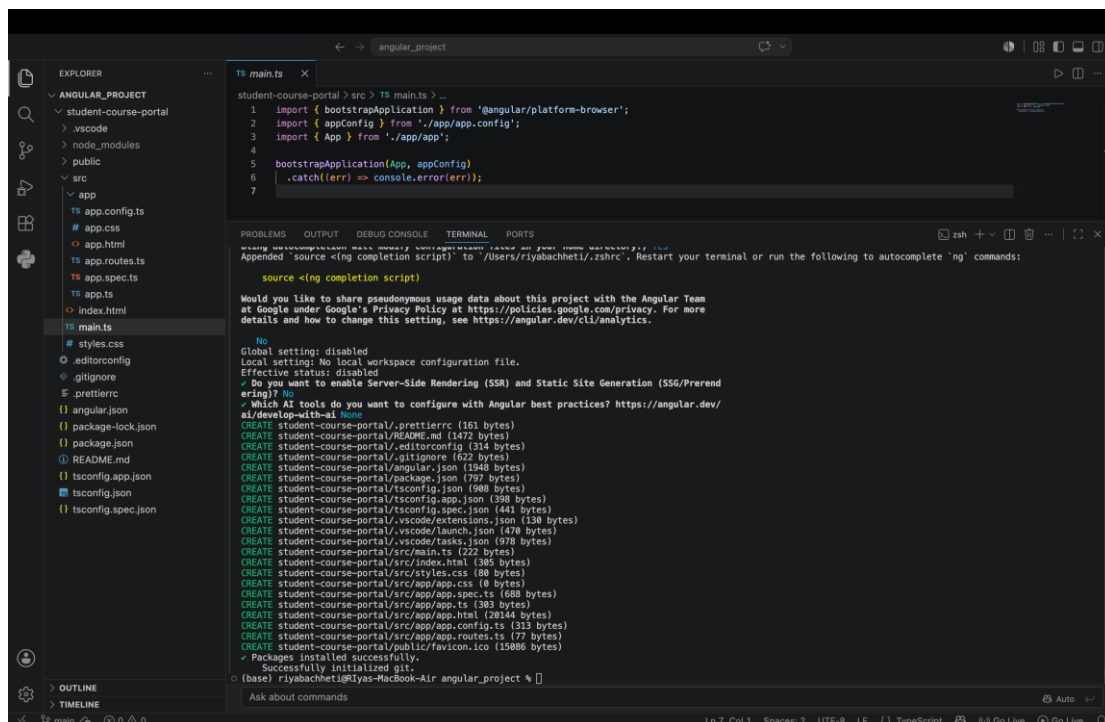
❖ Environment Setup, Project Structure & First Component

Scaffold and Explore the Angular Project

Goal: Create the Angular workspace and understand what each generated file does.

Steps:

1. Run `ng new student-course-portal --routing --style=css`. Accept defaults for all prompts.



The screenshot shows the Visual Studio Code interface with the Angular project 'student-course-portal' open. The Explorer sidebar on the left displays the project structure, including files like `angular.json`, `tsconfig.json`, `package.json`, `src/main.ts`, `src/app/app.module.ts`, `src/app/app.component.ts`, and `src/index.html`. The main editor shows the `src/main.ts` file with the following code:

```
1 import { bootstrapApplication } from '@angular/platform-browser';
2 import { appConfig } from './app/app.config';
3 import { App } from './app/app';
4
5 bootstrapApplication(App, appConfig)
6   .catch((err) => console.error(err));
7
```

The terminal at the bottom shows the output of the `ng new` command, including prompts for sharing usage data, enabling SSR, and configuring AI tools, all of which were accepted or disabled. It also lists the files created and the successful installation of packages and git initialization.

2. Open the project in VS Code. For each of the following files, write a one-line comment in a `notes.txt` file explaining its purpose: `angular.json`, `tsconfig.json`, `tsconfig.app.json`, `package.json`, `src/main.ts`, `src/app/app.module.ts` (or `app.config.ts` in standalone mode), `src/app/app.component.ts`, `src/index.html`.

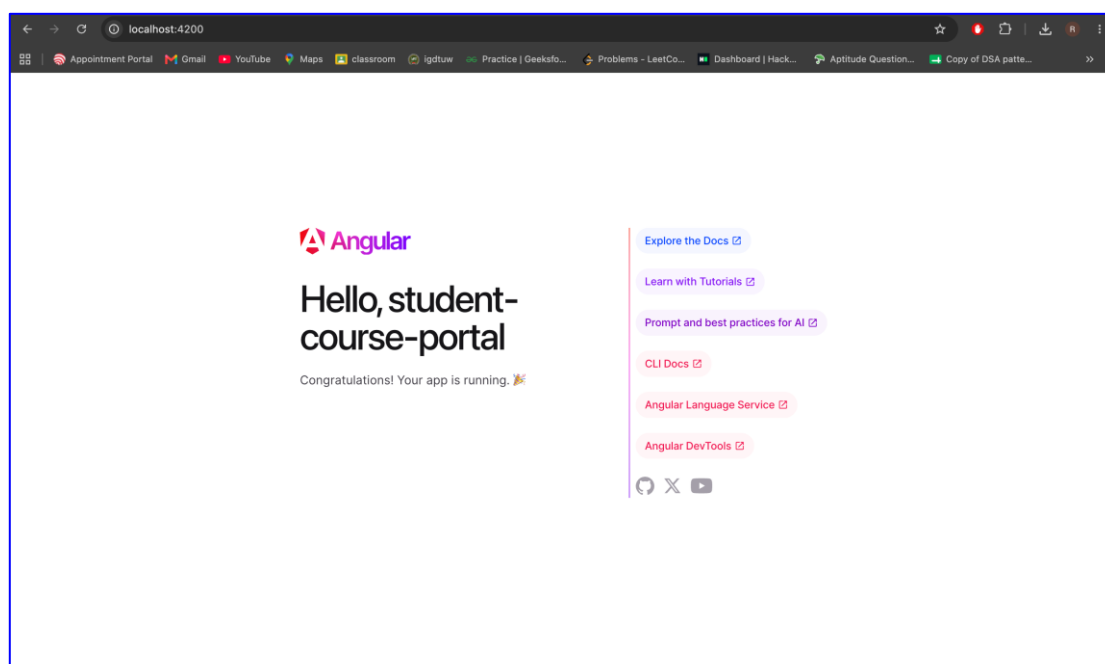
```
notes.txt U X
student-course-portal > notes.txt
1  angular.json
2  Configures how Angular builds, serves, tests, and manages the project.
3
4  tsconfig.json
5  Contains the main TypeScript compiler configuration for the project.
6
7  tsconfig.app.json
8  Contains TypeScript compiler settings specific to the Angular application source code.
9
10 package.json
11 Stores project metadata, dependencies, and npm scripts used to run the application.
12
13 src/main.ts
14 The entry point of the Angular application that bootstraps the root component.
15
16 src/app/app.config.ts
17 Configures application-wide providers and settings for the standalone Angular application.
18
19 src/app/app.ts
20 Defines the root component that controls the main user interface of the application.
21
22 src/index.html
23 The main HTML page that hosts the Angular application.
24
```

3. Run `ng serve` and open `http://localhost:4200`. Verify the default Angular welcome page loads.

```
Successfully initialized git.
● (base) riyabachheti@Riyas-MacBook-Air angular_project % cd student-course-portal
○ (base) riyabachheti@Riyas-MacBook-Air student-course-portal % ng serve
Initial chunk files | Names | Raw size
main.js | main | 47.84 kB
styles.css | styles | 95 bytes
| Initial total | 47.94 kB

Application bundle generation complete. [0.939 seconds] - 2026-07-24T08:59:01.594Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
→ Local: http://localhost:4200/
→ press h + enter to show help
```



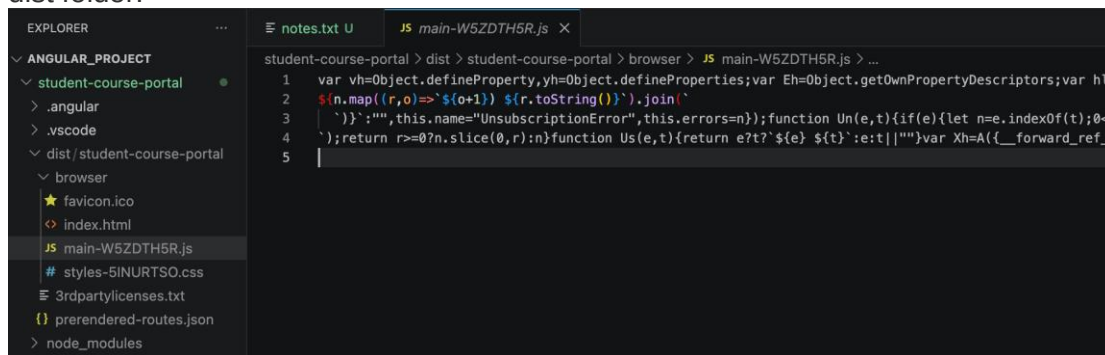
4. Run `ng build` and inspect the `dist/` folder. Note the generated files — identify which file contains the compiled application code.

```
(base) riyabachheti@RIYas-MacBook-Air student-course-portal % ng build
Initial chunk files | Names | Raw size | Estimated transfer size
main-W5ZDTH5R.js | main | 214.73 kB | 58.91 kB
styles-5INURTS0.css | styles | 0 bytes | 0 bytes
Initial total | 214.73 kB | 58.91 kB

Application bundle generation complete. [1.376 seconds] - 2026-07-24T09:03:39.105Z

Output location: /Users/riyabachheti/Documents/Coding/angular_project/student-course-portal/dist/student-course-portal
```

The compiled application code is contained in the generated `main-*.js` file inside the `dist` folder.



```
student-course-portal > dist > student-course-portal > browser > JS main-W5ZDTH5R.js > ...
1 var vh=Object.defineProperty,yh=Object.defineProperties;var Eh=Object.getOwnPropertyDescriptors;var hl=
2 $(n.map((r,o)=>`${o+1}) ${r.toString()}`).join('
3 `)`:'"',this.name="UnsubscriptionError",this.errors=n));function Un(e,t){if(e){let n=e.indexOf(t);0<=
4 `);return r==0?n.slice(0,r):n}function Us(e,t){return e?t?`${e} ${t}`:e:t||''}var Xh=A({__forward_ref_
5 |
```

5. In `angular.json`, locate the budgets configuration under `architect > build > configurations > production`.

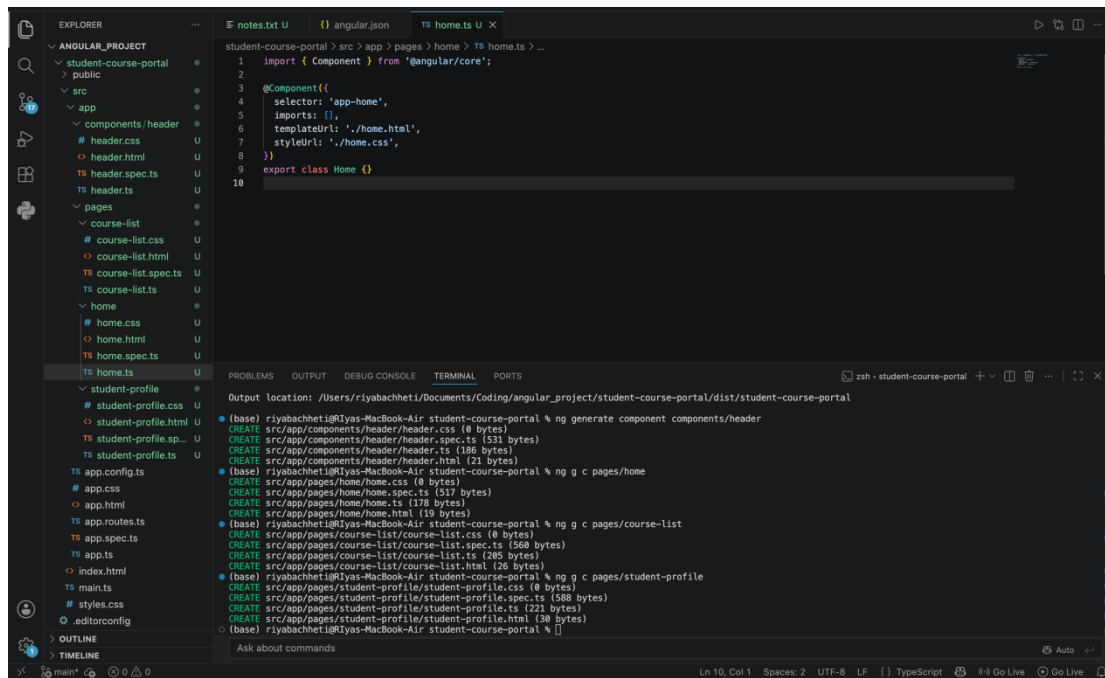
```
notes.txt U {} angular.json X
-portal > {} angular.json > {} projects > {} student-course-portal > {} architect > {} build >
8   "projects": {
9     "student-course-portal": {
15      "architect": {
16        "build": {
31          "configurations": {
32            "production": {
33              "budgets": [
34                {
35                  "type": "initial",
36                  "maximumWarning": "500kB",
37                  "maximumError": "1MB"
38                },
39                {
40                  "type": "anyComponentStyle",
41                  "maximumWarning": "4kB",
42                  "maximumError": "8kB"
43                }
44              ],
45              "outputHashing": "all"
46            },
47            "development": {
48              "optimization": false,
49              "extractLicenses": false,
50              "sourceMap": true
          }
        }
      }
    }
  }
```

Task 2: Create and Organise Components

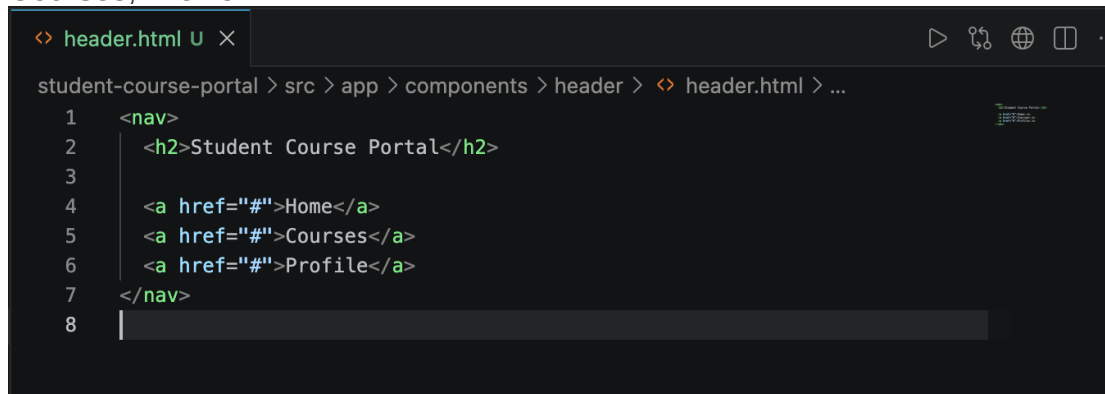
Goal: Generate the core page components for the Student Course Portal.

Steps:

6. Generate four components using Angular CLI: ng generate component components/header, ng generate component pages/home, ng generate component pages/course-list, ng generate component pages/student-profile. Observe the 4 files generated for each: .ts, .html, .css, .spec.ts.



7. In HeaderComponent's template (header.component.html), add a `<nav>` with the portal name 'Student Course Portal' and placeholder links: Home, Courses, Profile.



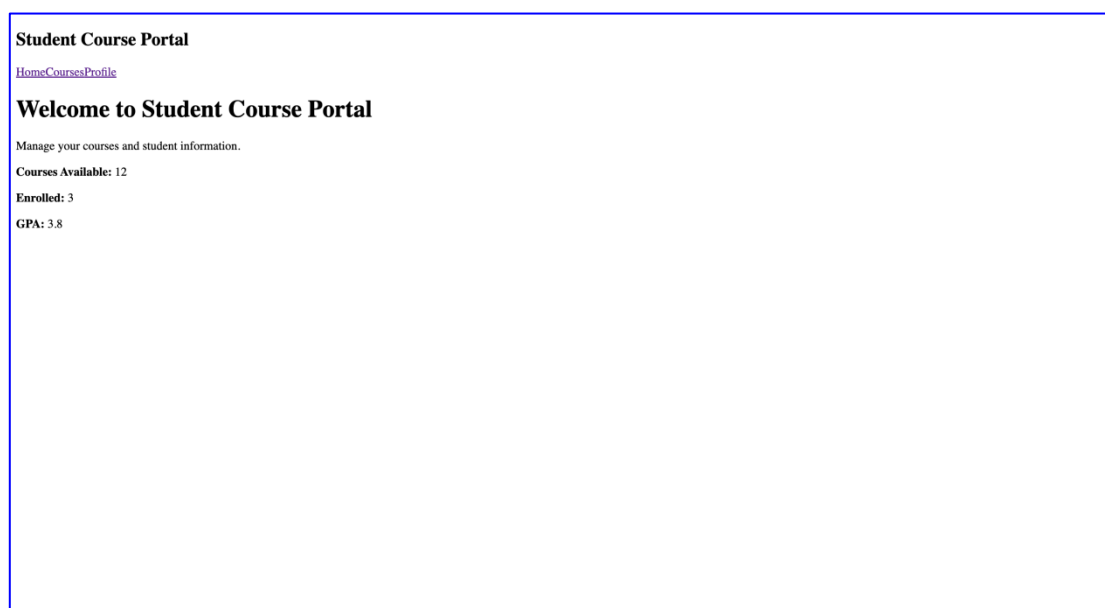
8. In HomeComponent's template, add a welcome heading (`<h1>`), a short description paragraph, and a stats row showing three hardcoded values: Courses Available: 12, Enrolled: 3, GPA: 3.8.

```
<> home.html U X
student-course-portal > src > app > pages > home > <> home.html > ...
1  <h1>Welcome to Student Course Portal</h1>
2
3  <p>
4    Manage your courses and student information.
5  </p>
6
7  <div>
8    <p><strong>Courses Available:</strong> 12</p>
9    <p><strong>Enrolled:</strong> 3</p>
10   <p><strong>GPA:</strong> 3.8</p>
11 </div>
12
13
```

9. Add `<app-header>` and `<router-outlet>` to `app.component.html`, replacing the default boilerplate content.

```
<> app.html M X
student-course-portal > src > app > <> app.html > ...
> Find Aa ab .* No results ↑ ↓ ≡ ×
1  <app-header></app-header>
2
3  <router-outlet></router-outlet>
4
5
```

10. Verify ng serve still compiles without errors and the header and home page content are visible.



❖ Data Binding, Lifecycle Hooks & Component Communication

Task 1: All Four Binding Types

Goal: Practise interpolation, property binding, event binding, and two-way binding in one component.

Steps:

```
// Property binding [property] sends data from the component to the DOM.
// Two-way binding [(ngModel)] keeps the component and the DOM synchronized in both directions.
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-home',
  imports: [FormsModule],
  templateUrl: './home.html',
  styleUrls: ['./home.css'],
})
export class Home {

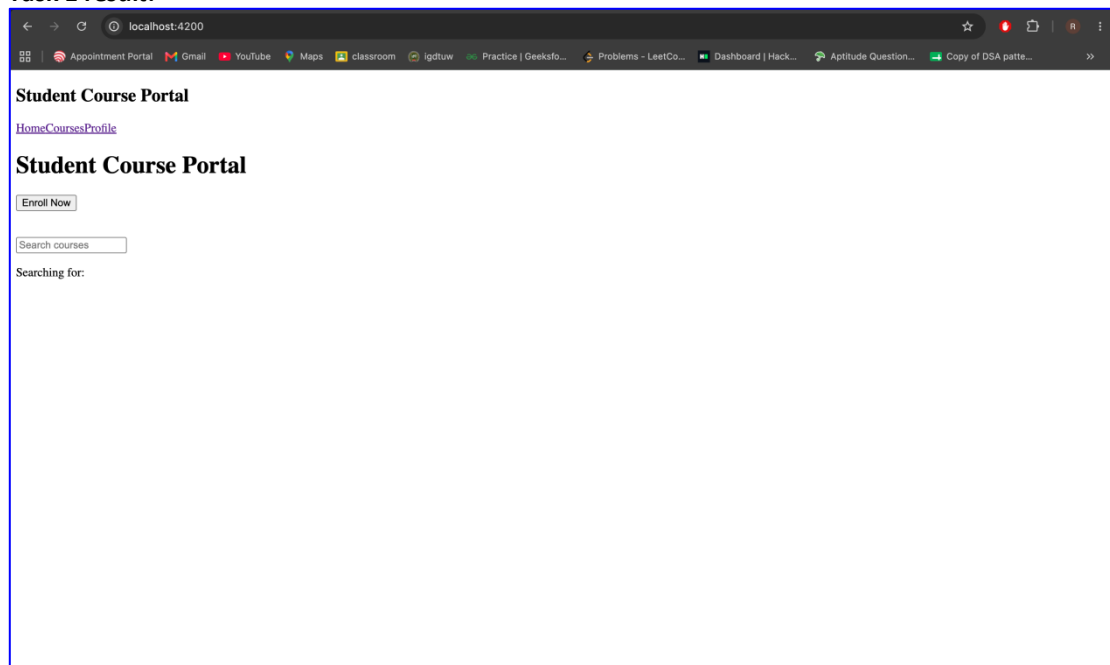
  portalName = 'Student Course Portal';

  isPortalActive = true;

  message = '';
  searchTerm = '';

  onEnrollClick() {
    this.message = 'Enrollment opened!';
  }
}
```

Task 1 result:



Task 2: Lifecycle Hooks

Goal: Implement the three most important lifecycle hooks and observe when they fire.

Task 2 result:

Student Course Portal

[Home](#)[Courses](#)[Profile](#)

Java Programming

Code: CS101

Credits: 4

Angular

Code: CS201

Credits: 3

Spring Boot

Code: CS301

Credits: 4

Angular is running in development mode.

Course input changed

Previous: undefined

Current: {id: 1, name: 'Java Programming', code: 'CS101', credits: 4}

Course input changed

Previous: undefined

Current: {id: 2, name: 'Angular', code: 'CS201', credits: 3}

Course input changed

Previous: undefined

Current: {id: 3, name: 'Spring Boot', code: 'CS301', credits: 4}

Task 3: @Input and @Output — Parent-Child Communication

Goal: Pass data down with @Input and emit events up with @Output.

Task 3 result:

Student Course Portal

[Home](#)[Courses](#)[Profile](#)

Java Programming

ID: 1

Code: CS101

Credits: 4

Enroll

Angular

ID: 2

Code: CS201

Credits: 3

Enroll

Spring Boot

ID: 3

Code: CS301

Credits: 4

Enroll

Database Systems

ID: 4

Code: CS401

Credits: 3

Enroll

Operating Systems

ID: 5

Code: CS501

Credits: 4

Enroll

Selected course ID: 5

Directives & Pipes — Built-in and Custom

Task : Structural Directives

Goal: Control DOM structure dynamically using *ngIf, *ngFor, and *ngSwitch.

```
course-card.html U X TS course-list.ts U TS course-card.ts U course-list.html U home.html U
student-course-portal > src > app > components > course-card > course-card.html > ...
1 <h3>{{ course.name }}</h3>
2
3 <p><strong>ID:</strong> {{ course.id }}</p>
4
5 <p><strong>Code:</strong> {{ course.code }}</p>
6
7 <p><strong>Credits:</strong> {{ course.credits }}</p>
8
9 <div [ngSwitch]="course.gradeStatus">
10
11 <p *ngSwitchCase="'passed'" style="color: green;">
12 | Passed
13 </p>
14
15 <p *ngSwitchCase="'failed'" style="color: red;">
16 | Failed
17 </p>
18
19 <p *ngSwitchDefault style="color: gray;">
20 | Pending
21 </p>
22
23 </div>
24
25 <button (click)="enrollRequested.emit(course.id)">
26 | Enroll
27 </button>
28
29 <hr>
```

```
course-list.html U X course-card.html U # course-card.c: course-card.c: course-card.c:
student-course-portal > src > app > pages > course-list > course-list.html > ...
1 <div *ngIf="isLoading">
2 | <p>Loading courses...</p>
3 </div>
4
5 <div *ngIf="!isLoading">
6
7 <div *ngIf="courses.length > 0; else noCourses">
8
9 <app-course-card
10 | *ngFor="let c of courses; let i = index; trackBy: trackBy(courseId)"
11 | [course]="c"
12 | (enrollRequested)="onEnroll($event)">
13 </app-course-card>
14
15 <p *ngIf="selectedCourseId">
16 | Selected course ID: {{ selectedCourseId }}
17 </p>
18
19 </div>
20
21 </div>
22
23 <ng-template #noCourses>
24 | <p>No courses available.</p>
25 </ng-template>
26
```

Student Course Portal

[Home](#)[Courses](#)[Profile](#)

Java Programming

ID: 1

Code: CS101

Credits: 4

Passed

Enroll

Angular

ID: 2

Code: CS201

Credits: 3

Failed

Enroll

Spring Boot

ID: 3

Code: CS301

Credits: 4

Pending

Enroll

Database Systems

ID: 4